



**POLITECNICO  
MILANO 1863**

**COMPUTER SCIENCE AND ENGINEERING  
INTERNET OF THINGS ( IoT )  
2025 - 2026**

# **Challenge #3**

## **Node-Red & LoRaWAN**

**Team:**

Elena Lin (Leader) 10802468

Jiayi Zheng 10811585

# Content

|                                            |    |
|--------------------------------------------|----|
| 1. Introduction.....                       | 3  |
| 1.1 Flow Overview.....                     | 3  |
| 2. Initialization.....                     | 4  |
| 2.1 Component Detail.....                  | 4  |
| 3. ID Generation And Publication.....      | 6  |
| 3.1 Component Detail.....                  | 6  |
| 4. Message Processing And Filtering.....   | 8  |
| 4.1 Component Detail.....                  | 8  |
| 4.2 Real-Time Dashboard Visualization..... | 14 |
| 5. ZigBee Link Cost Processing.....        | 15 |
| 5.1 Component Detail.....                  | 16 |
| 5.2 ThingSpeak Output Visualization.....   | 19 |

# Part 1| Challenge - NodeRed

## 1. Introduction

This report presents the implementation of Challenge 3 using Node-Red, MQTT communication, and ThingSpeak integration.

The objective of this challenge is to build a complete IoT data processing pipeline capable of:

- generating random IDs
- publishing MQTT messages
- processing CSV packet data
- filtering ZigBee packets
- extracting electrical measurements
- visualizing real-time data
- integrating with Thingspeak cloud services

The system is implemented using multiple interconnected Node-Red branches, where each branch performs a dedicated task inside the overall processing workflow.

According to the assignment requirements, the flow must generate MQTT messages periodically, process the *challenge3.csv* dataset, extract ZigBee information, apply rate limiting, and stop execution after exactly 200 received MQTT messages.

### 1.1 Flow Overview

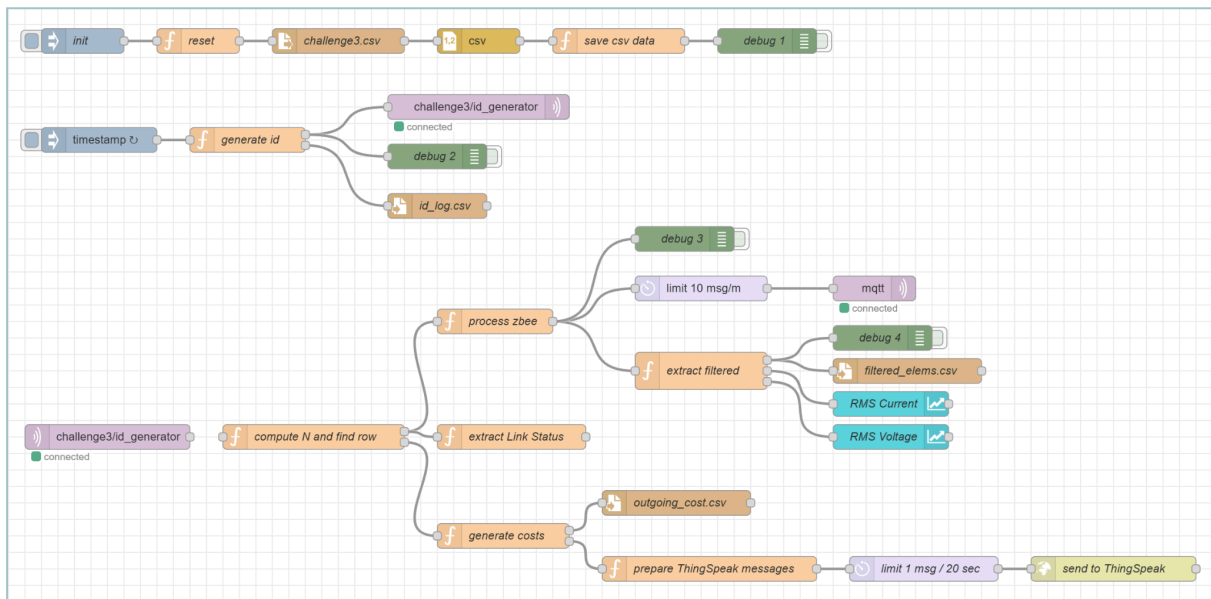


Figure 1: Complete Node-Red flow implementation

## 2. Initialization

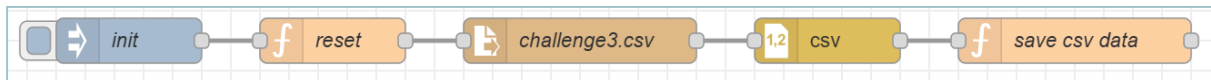


Figure 2: Initialization Branch

The initialization phase is separated from the runtime processing flow in order to improve efficiency and simplify data management.

The purpose of this flow is to prepare all the required data structures and variables before starting the MQTT processing phase.

During initialization, the system resets the internal counters and loads the *challenge3.csv* dataset into memory for packet lookup operations.

The parsed CSV content is then stored inside the flow context, allowing the other processing branches to access the dataset during runtime without repeatedly reading the file from disk, reducing unnecessary I/O operations and improving the overall system performance.

| NODE                  | DESCRIPTION                                                                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>init</i>           | This <b>inject node</b> starts the initialization phase. It is manually triggered before running the main flow.                                     |
| <i>reset</i>          | This <b>function node</b> resets the flow context variables used during the execution. This ensures that every execution starts from a clean state. |
| <i>challenge3.csv</i> | This <b>file node</b> reads the input file <i>challenge3.csv</i> from the local file system.                                                        |
| <i>csv</i>            | This <b>CSV node</b> converts the raw CSV text into JavaScript objects.                                                                             |
| <i>save csv data</i>  | This <b>function node</b> stores the parsed CSV array inside the flow context.                                                                      |

### 2.1 Component Detail

#### ➤ reset

Name

reset

Setup

On Start

On Message

On Stop

```
1 //initialize the counter for ID messages (used when writing id_log.csv)
2 flow.set("id_no", 0);
3
4 //initialize how many MQTT messages have been received (limit = 200)
5 flow.set("received_count", 0);
6
7 //initialize row counter for filtered_elems.csv
8 flow.set("filtered_no", 0);
9
10 //initialize row counter for outgoing_cost.csv
11 flow.set("outgoing_no", 0);
12
13 //initialize an empty object to store Links Status costs (Source -> Destination -> Cost)
14 flow.set("costs", {});
15
16 //clear previously stored CSV data (avoid using old data by mistake)
17 flow.set("csv_data", null);
18
19 msg.payload = "reset done";
20 return msg;
```

| VARIABLE              | DESCRIPTION                                                                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>id_no</i>          | Stores the progressive row number used when writing generated IDs into <i>id_log.csv</i> .                                                                   |
| <i>received_count</i> | Counts how many MQTT messages have been received and processed by the subscriber flow.                                                                       |
| <i>filtered_no</i>    | Stores the progressive row number used when writing filtered ZBEE attribute data into <i>filtered_elems.csv</i> .                                            |
| <i>outgoing_no</i>    | Stores the progressive row number used for the generation of <i>outgoing_cost.csv</i> .                                                                      |
| <i>costs</i>          | Is initialized as an empty JavaScript object.<br>It is later used to store Link Status information.                                                          |
| <i>csv_data</i>       | Stores the parsed content of <i>challenge3.csv</i> .<br>It is reset to null during initialization to avoid using outdated CSV data from previous executions. |

### ➤ save csv data

Name

Setup
On Start
On Message
On Stop

```

1 //save the CSV data (an array of rows) into flow context
2 flow.set("csv_data", msg.payload);
3
4 //print the number of rows loaded (should be 5218 if correct)
5 node.warn("CSV loaded: " + msg.payload.length);
6
7 return msg;

```

### 3. ID Generation And Publication

This branch is responsible for periodically generating MQTT messages.

Each generated message is published through MQTT on the topic *challenge3/id\_generator* and stored inside the *id\_log.csv* file.

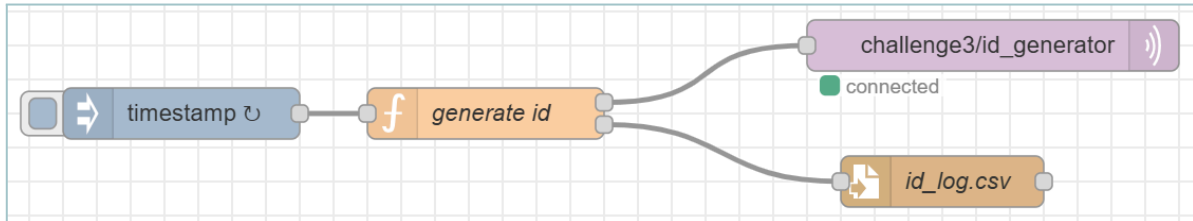


Figure 3: ID Generation and Publication branch

| NODE                           | DESCRIPTION                                                                                                                                                               |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>timestamp</i>               | This <b>inject node</b> periodically triggers the flow execution.                                                                                                         |
| <i>generate id</i>             | This <b>function node</b> generates random IDs.<br>Two different outputs are generated:<br>1. an MQTT payload<br>2. a CSV-formatted string written into <i>id_log.csv</i> |
| <i>challenge3/id_generator</i> | This <b>MQTT output node</b> publishes the generated payload to the local MQTT broker using the topic <i>challenge3/id_generator</i> .                                    |
| <i>id_log.csv</i>              | This <b>file node</b> stores the generated IDs and timestamps into the file <i>id_log.csv</i> .                                                                           |

#### 3.1 Component Detail

##### ➤ timestamp

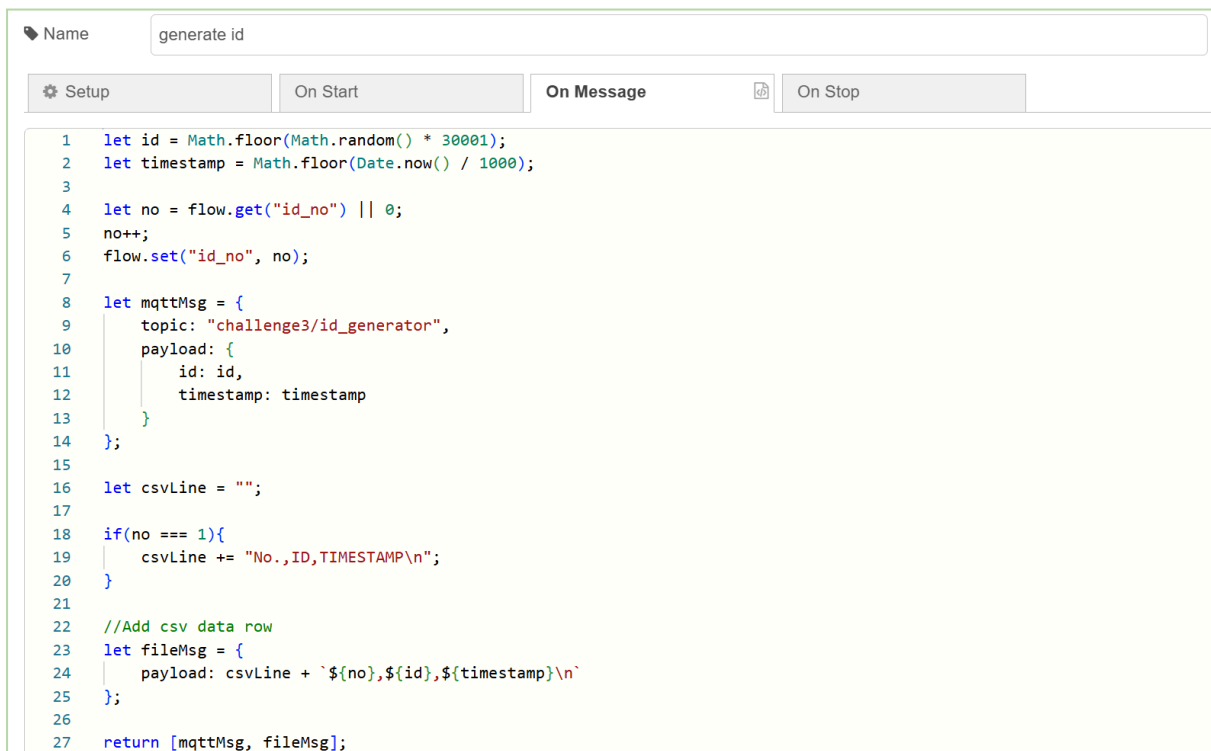
🔄 Repeat

interval

every 1 seconds

According to the assignment requirements, this inject node is configured to generate exactly one message every second.

## ➤ generated id



```
1 let id = Math.floor(Math.random() * 30001);
2 let timestamp = Math.floor(Date.now() / 1000);
3
4 let no = flow.get("id_no") || 0;
5 no++;
6 flow.set("id_no", no);
7
8 let mqttMsg = {
9   topic: "challenge3/id_generator",
10   payload: {
11     id: id,
12     timestamp: timestamp
13   }
14 };
15
16 let csvLine = "";
17
18 if(no === 1){
19   csvLine += "No.,ID,TIMESTAMP\n";
20 }
21
22 //Add csv data row
23 let fileMsg = {
24   payload: csvLine + `${no},${id},${timestamp}\n`
25 };
26
27 return [mqttMsg, fileMsg];
```

The MQTT message payload has the following structure:

{“id”: random number, “timestamp”: current time in seconds}

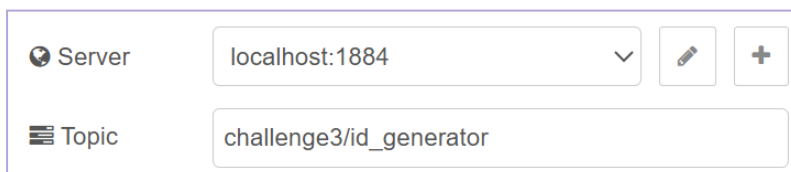
The formatted CSV line has the following structure:

No., ID, TIMESTAMP

where No. is the incremental row number.

## ➤ challenge3/id\_generator

MQTT messages are published to the topic *challenge3/id\_generator*.



Server: localhost:1884

Topic: challenge3/id\_generator

## 4. Message Processing And Filtering

This branch is responsible for subscribing to the MQTT messages generated by the ID generation branch and processing the corresponding packets retrieved from the *challenge3.csv* dataset.

For each received MQTT message, the system computes  $N$  and searches the previously loaded CSV dataset for the matching packet.

The retrieved packets are then filtered in order to identify ZigBee ZCL messages.

Relevant ZigBee information is extracted, republished through MQTT with rate limiting applied, and further processed to obtain specific measurements.

The extracted measurements are stored inside *filtered\_elems.csv*, while RMS Current and RMS Voltage values are simultaneously visualized in real time using Node-Red dashboard charts.

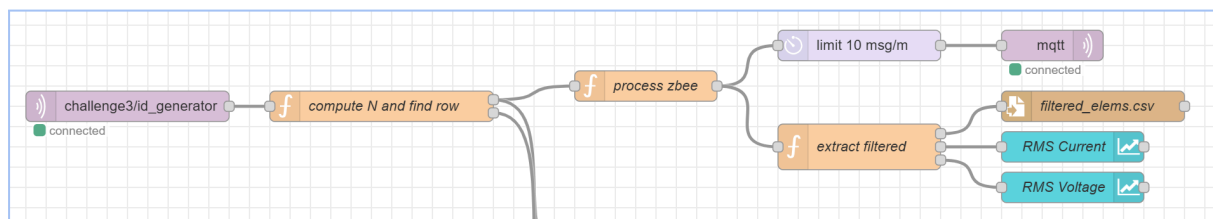


Figure 4: Message Processing and Filtering Branch

| NODE                           | DESCRIPTION                                                                                      |
|--------------------------------|--------------------------------------------------------------------------------------------------|
| <i>challenge3/id_generator</i> | This <b>MQTT input node</b> subscribes to MQTT messages containing generated IDs.                |
| <i>compute N and find row</i>  | This <b>function node</b> computes $N$ and retrieves the corresponding CSV row.                  |
| <i>process zbee</i>            | This <b>function node</b> filters packets containing ZBEE_ZCL and prepares MQTT output messages. |
| <i>limit 10 msg/m</i>          | This <b>delay node</b> limits MQTT publishing rate.                                              |
| <i>mqtt</i>                    | This <b>MQTT output node</b> publishes processed ZigBee messages.                                |
| <i>extract filtered</i>        | This <b>function node</b> extracts Active Power, RMS Current, and RMS Voltage values.            |
| <i>filtered_elems.csv</i>      | This <b>file node</b> appends filtered measurements to the CSV file.                             |
| <i>RMS Current</i>             | This <b>Dashboard Chart</b> displays RMS Current values in real time.                            |
| <i>RMS Voltage</i>             | This <b>Dashboard Chart</b> displays RMS Voltage values in real time.                            |

### 4.1 Component Detail

#### ➤ challenge3/id\_generator

The MQTT subscriber listens to the topic *challenge3/id\_generator*.

Each incoming message contains the generated ID and timestamp.



Server

localhost:1884

+

Action

Subscribe to single topic

Topic

challenge3/id\_generator

## ➤ compute N and find row

Name

compute N and find row

Setup

On Start

On Message

On Stop

```

1  let rows = flow.get("csv_data");//retrieve csv data previously stored in flow context
2  if (!rows || rows.length === 0) {
3    |   node.error("CSV not loaded. Click init first!");
4    |   return null;
5  }
6
7  let payload = msg.payload;
8  if (typeof payload === "string") payload = JSON.parse(payload);// if string -> JSON
9
10 let id = Number(payload.id);//extract id from mqtt payload
11 //check id
12 if (isNaN(id)) {
13   |   node.warn("Invalid ID payload: " + JSON.stringify(msg.payload));
14   |   return null;
15 }
16
17 //count received mqtt messages
18 let count = flow.get("received_count") || 0;
19 count++;
20 flow.set("received_count", count);
21
22 if (count > 200) return null;// do not process more than 200 messages
23
24 let N = id % 5218;//compute N
25
26
27
28
29
30
31
32 msg.sub_id = id;
33 msg.N = N;
34 msg.row = row;
35 msg.payload = row;
36
37 node.warn("ID=" + id + " → N=" + N + " count=" + count);
38
39 if (count === 200) {
40   |   let endMsg = {
41   |     |   payload: "generate outgoing_cost.csv"
42   |   };
43   |   //output 1 -> send the processed mqtt message (msg)
44   |   //output 2 -> trigger generate outgoing_cost.csv
45   |   return [msg, endMsg];
46 }
47
48 //output 1 -> send the processed mqtt message (msg)
49 //output 2 -> send nothing (null) until the 200th message is reached
50 return [msg, null];

```

For each received message, the flow computes  $N = ID \bmod 5218$ .

The computed value  $N$  is used to search the previously loaded *challenge3.csv* dataset and retrieve the corresponding packet whose *Packet Number* matches  $N$ .

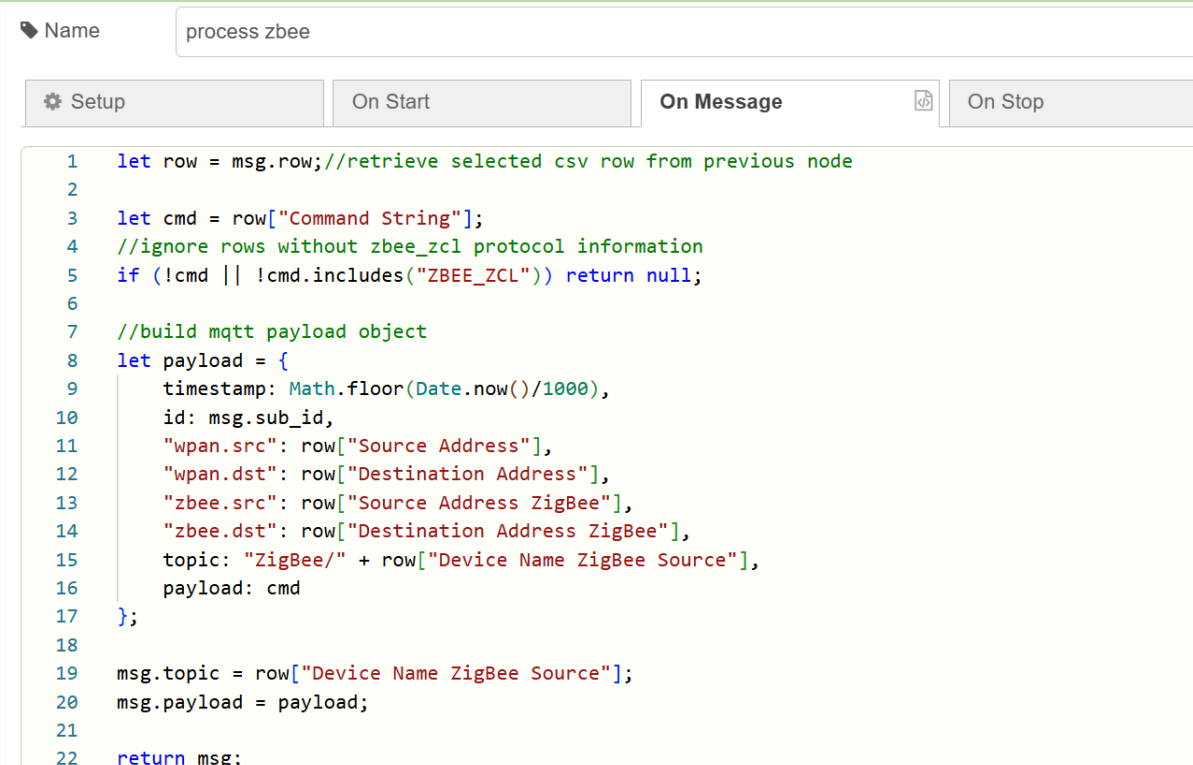
According to the assignment requirements, the flow stops working after receiving exactly 200 ID messages from the subscription.

The matched row is then sent to the next processing nodes.

One output is used for ZigBee message processing, while another output is used to generate *outgoing\_cost.csv* and prepare ThingSpeak updates after the 200-message limit is reached (detailed explanation in section 5).

If no matching packet is found, or if the limit of 200 messages has already been reached, the node returns *null* and stops processing that message.

### ➤ process zbee



The screenshot shows the configuration for a node named "process zbee". The "On Message" tab is active, displaying the following JavaScript code:

```
1 let row = msg.row; //retrieve selected csv row from previous node
2
3 let cmd = row["Command String"];
4 //ignore rows without zbee_zcl protocol information
5 if (!cmd || !cmd.includes("ZBEE_ZCL")) return null;
6
7 //build mqtt payload object
8 let payload = {
9   timestamp: Math.floor(Date.now()/1000),
10   id: msg.sub_id,
11   "wpan.src": row["Source Address"],
12   "wpan.dst": row["Destination Address"],
13   "zbee.src": row["Source Address ZigBee"],
14   "zbee.dst": row["Destination Address ZigBee"],
15   topic: "ZigBee/" + row["Device Name ZigBee Source"],
16   payload: cmd
17 };
18
19 msg.topic = row["Device Name ZigBee Source"];
20 msg.payload = payload;
21
22 return msg;
```

After retrieving the packet, the system processes the packet content in order to identify ZigBee ZCL messages.

Only packets containing a *ZBEE\_ZCL* layer are processed and republished through MQTT.

This filtering stage reduces unnecessary traffic and focuses only on ZigBee application-layer messages.

### ➤ limit 10 msg/m

⌵ Action Rate Limit ⌵

All messages ⌵

⌚ Rate 10 ⌵ msg(s) per 1 ⌵ Minute ⌵

☐ allow msg.rate (in ms) to override rate

Queue intermediate messages ⌵

A delay node configured as a rate limiter restricts MQTT publishing to a maximum of 10 messages per minute.

Queue mode is enabled so that intermediate messages are buffered instead of discarded.

### ➤ mqtt

🌐 Server localhost:1884 ⌵ ✎ +

⌵ Topic Topic

When a valid ZigBee packet is identified, a new MQTT message is generated.

The MQTT topic is dynamically assigned by the previous Function node using *msg.topic*.

## ➤ extract filtered

 Name

⚙ Setup

On Start

On Message 

On Stop

```
1 //retrieve command string and timestamp from incoming mqtt payload
2 let cmd = msg.payload.payload;
3 let timestamp = msg.payload.timestamp;
4 if (!cmd) return null; //stop processing if command string is missing
5
6 //attributes required by the assignment
7 let interested = ["Active Power", "RMS Current", "RMS Voltage"];
8
9 let seq = "";
10 let attrs = [];
11 let statuses = [];
12 let types = [];
13 let values = [];
14
15 //extract information from the zbee payload using regular expressions
16 try {
17     //extract sequence number
18     let seqMatch = cmd.match(/'Sequence Number': '([^']+)'\/);
19     if (seqMatch) seq = seqMatch[1];
20     //extract attribute list
21     let attrMatch = cmd.match(/'Attribute': \[([^\]]+)\]\/);
22     if (attrMatch) {
23         attrs = attrMatch[1].split(",").map(x => x.replace(/'/g, "").trim());
24     }
25     //extract status list
26     let statusMatch = cmd.match(/'Status': \[([^\]]+)\]\/);
27     if (statusMatch) {
28         statuses = statusMatch[1].split(",").map(x => x.replace(/'/g, "").trim());
29     }
30     //extract data type list
31     let typeMatch = cmd.match(/'Data Type': \[([^\]]+)\]\/);
32     if (typeMatch) {
33         types = typeMatch[1].split(",").map(x => x.replace(/'/g, "").trim());
34     }
35     //extract measurement values
36     let intMatch = cmd.match(/'Int16': '([^']+)'\/);
37     let uintMatch = cmd.match(/'UInt16': \[([^\]]+)\]\/);
38
39     if (intMatch) values.push(intMatch[1]);
40
41     if (uintMatch) {
42         let u = uintMatch[1].split(",").map(x => x.replace(/'/g, "").trim());
43         values = values.concat(u);
44     }
45 } catch(e) {
46     node.warn("Parse error: " + e.message);
47     return null;
48 }
```

```

49
50 let csvLines = [];
51 let currentMsg = null;
52 let voltageMsg = null;
53
54 //process all extracted attributes
55 for (let i = 0; i < attrs.length; i++) {
56     let attrName = attrs[i].split("(")[0].trim();
57     //keep only required measurements
58     if (!interested.includes(attrName)) continue;
59
60     let status = (statuses[i] || "").split("(")[0].trim();
61     let dtype = (types[i] || "").split("(")[0].trim();
62     let valueRaw = values[i] || "";
63     let value = valueRaw.split("(")[0].trim();
64     //skip incomplete entries
65     if (!status || !dtype || value === "") {
66         continue;
67     }
68
69     let no = flow.get("filtered_no") || 0;
70
71     if (no === 0) {
72         csvLines.push(
73             "No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value\n"
74         );
75     }
76
77     no++;
78     flow.set("filtered_no", no);
79
80     csvLines.push(`${no},${timestamp},${seq},${attrName},${status},${dtype},${value}\n`);
81
82     if (attrName === "RMS Current") {
83         currentMsg = {
84             topic: "RMS Current",
85             payload: Number(value),
86             timestamp: Date.now()
87         };
88     }
89
90     if (attrName === "RMS Voltage") {
91         voltageMsg = {
92             topic: "RMS Voltage",
93             payload: Number(value),
94             timestamp: Date.now()
95         };
96     }
97 }
98
99 //stop if no valid measurements are found
100 if (csvLines.length === 0) return null;
101
102 let fileMsg = { payload: csvLines.join("") };
103 let debugMsg = { payload: csvLines.join("") };
104
105 //output 1 -> filtered_elems.csv file
106 //output 2 -> RMS Current chart
107 //output 3 -> RMS Voltage chart
108 return [fileMsg, currentMsg, voltageMsg];

```

After processing ZigBee packets, the flow extracts selected electrical measurements from the Command String field.

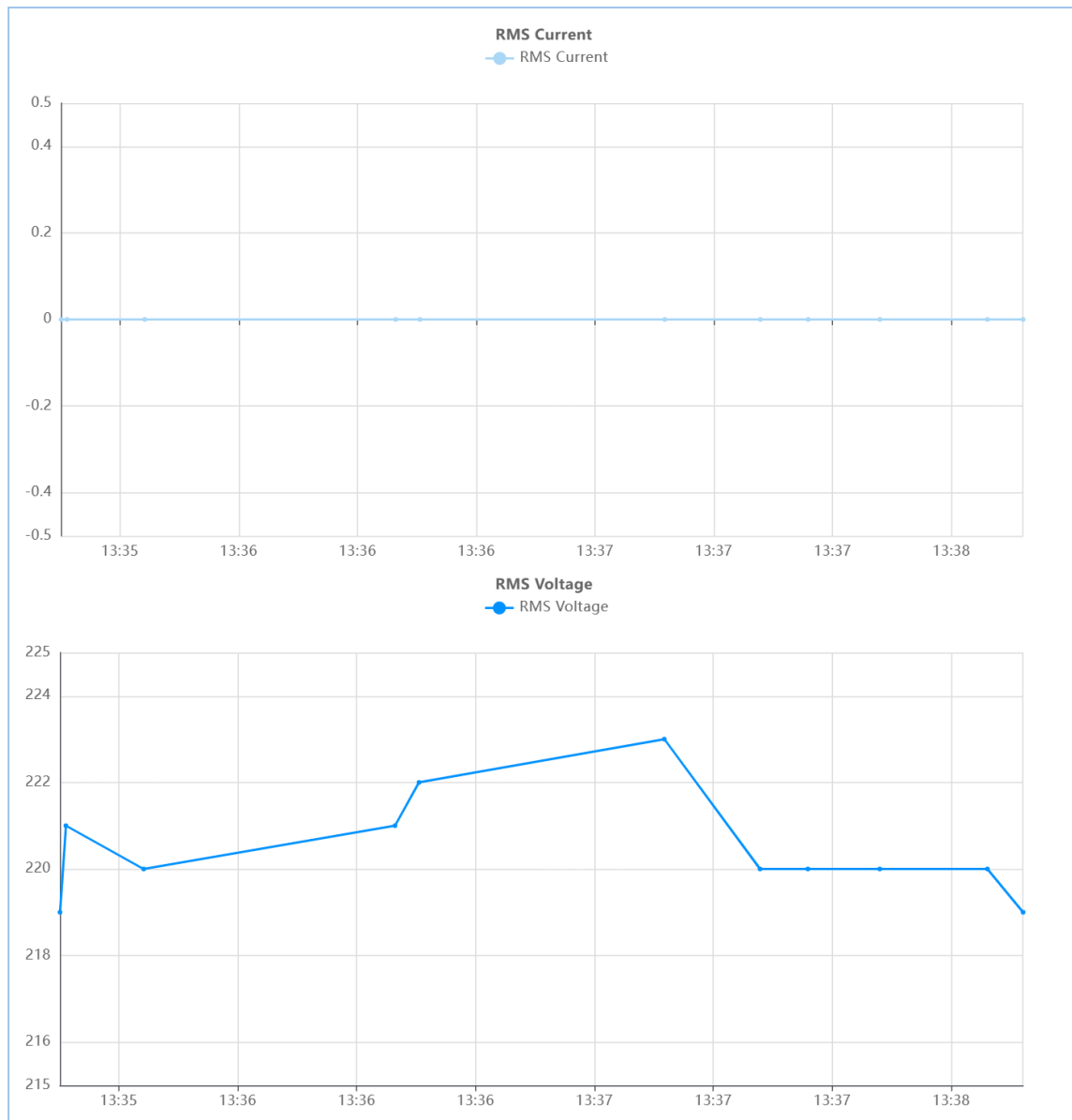
The extracted attributes include: Active Power, RMS Current, and RMS Voltage.

The extracted information is stored inside *filtered\_elems.csv* using the required CSV format.

## 4.2 Real-Time Dashboard Visualization

Two Node-Red dashboard charts are used to visualize RMS Current and RMS Voltage values in real time.

This allows continuous monitoring of the electrical measurements extracted from ZigBee packets.



| No. | Timestamp | Sequence | Attribute  | Status  | Data Type   | Data Value |
|-----|-----------|----------|------------|---------|-------------|------------|
| 1   | 1.78E+09  | 85       | Active Pow | Success | 16-Bit Sigr | 0          |
| 2   | 1.78E+09  | 85       | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 3   | 1.78E+09  | 85       | RMS Volta  | Success | 16-Bit Uns  | 219        |
| 4   | 1.78E+09  | 92       | Active Pow | Success | 16-Bit Sigr | 0          |
| 5   | 1.78E+09  | 92       | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 6   | 1.78E+09  | 92       | RMS Volta  | Success | 16-Bit Uns  | 221        |
| 7   | 1.78E+09  | 250      | Active Pow | Success | 16-Bit Sigr | 0          |
| 8   | 1.78E+09  | 250      | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 9   | 1.78E+09  | 250      | RMS Volta  | Success | 16-Bit Uns  | 220        |
| 10  | 1.78E+09  | 8        | Active Pow | Success | 16-Bit Sigr | 0          |
| 11  | 1.78E+09  | 8        | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 12  | 1.78E+09  | 8        | RMS Volta  | Success | 16-Bit Uns  | 221        |
| 13  | 1.78E+09  | 247      | Active Pow | Success | 16-Bit Sigr | 0          |
| 14  | 1.78E+09  | 247      | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 15  | 1.78E+09  | 247      | RMS Volta  | Success | 16-Bit Uns  | 222        |
| 16  | 1.78E+09  | 185      | Active Pow | Success | 16-Bit Sigr | 0          |
| 17  | 1.78E+09  | 185      | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 18  | 1.78E+09  | 185      | RMS Volta  | Success | 16-Bit Uns  | 223        |
| 19  | 1.78E+09  | 211      | Active Pow | Success | 16-Bit Sigr | 0          |
| 20  | 1.78E+09  | 211      | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 21  | 1.78E+09  | 211      | RMS Volta  | Success | 16-Bit Uns  | 220        |
| 22  | 1.78E+09  | 64       | Active Pow | Success | 16-Bit Sigr | 0          |
| 23  | 1.78E+09  | 64       | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 24  | 1.78E+09  | 64       | RMS Volta  | Success | 16-Bit Uns  | 220        |
| 25  | 1.78E+09  | 32       | Active Pow | Success | 16-Bit Sigr | 0          |
| 26  | 1.78E+09  | 32       | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 27  | 1.78E+09  | 32       | RMS Volta  | Success | 16-Bit Uns  | 220        |
| 28  | 1.78E+09  | 164      | Active Pow | Success | 16-Bit Sigr | 0          |
| 29  | 1.78E+09  | 164      | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 30  | 1.78E+09  | 164      | RMS Volta  | Success | 16-Bit Uns  | 220        |
| 31  | 1.78E+09  | 125      | Active Pow | Success | 16-Bit Sigr | 0          |
| 32  | 1.78E+09  | 125      | RMS Curre  | Success | 16-Bit Uns  | 0          |
| 33  | 1.78E+09  | 125      | RMS Volta  | Success | 16-Bit Uns  | 219        |

## 5. ZigBee Link Cost Processing

This branch receives MQTT ID messages, computes  $N = ID \bmod 5218$ , and retrieves the corresponding CSV row.

If the packet is a ZigBee Link Status message, the flow extracts and stores outgoing link costs by source and destination address.

After all 200 MQTT messages have been processed, the flow generates the `outgoing_cost.csv` file and sends the processed outgoing cost data to ThingSpeak with controlled transmission timing.

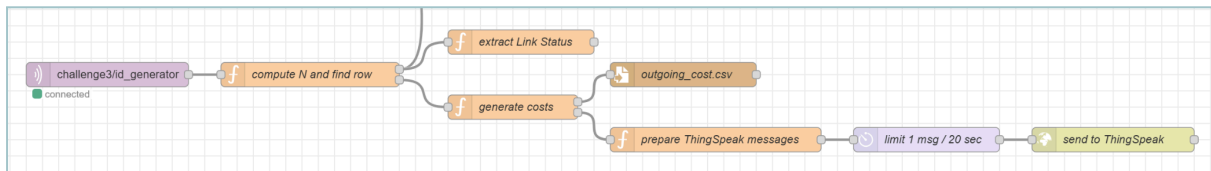


Figure 5: ZigBee Link Cost Processing Branch

The following table summarizes the functions and roles of each node used in Figure 5.

| NODE                               | DESCRIPTION                                                                                                                                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>extract Link Status</i>         | This <b>function node</b> checks for Link Status packets, extracts destination addresses and outgoing costs, and stores the cost data by source address.                           |
| <i>generate costs</i>              | This <b>function node</b> generates the <i>outgoing_cost.csv</i> data from the stored source, destination, and outgoing cost information.                                          |
| <i>outgoing_cost.csv</i>           | This <b>file node</b> writes generated outgoing cost data into the <i>outgoing_cost.csv</i> file using the required CSV format.                                                    |
| <i>prepare ThingSpeak messages</i> | This <b>function node</b> finds the smallest source address, sorts its destinations in ascending hexadecimal order, and prepares ThingSpeak HTTP messages with outgoing cost data. |
| <i>limit 1 msg/ 20 sec</i>         | This <b>delay node</b> controls the transmission rate by allowing only one ThingSpeak message to pass every 20 seconds.                                                            |
| <i>send to ThingSpeak</i>          | This <b>http request node</b> sends the prepared outgoing cost data to the ThingSpeak channel through the HTTP API.                                                                |



## 5.1 Component Detail

This section shows the setup and code of each Node-RED component in the processing branch.

### ➤ extract Link Status

The image shows the Node-RED component editor for a node named 'extract Link Status'. The editor has tabs for 'Setup', 'On Start', 'On Message', and 'On Stop'. The 'On Message' tab is selected, showing a JavaScript code editor with 50 lines of code. The code processes a message, extracts link status information from a command string, and updates a flow's 'costs' object. It includes comments for each step, such as 'ZigBee source address', 'Command String', and 'extract Links section from Command String'. The code uses JavaScript's 'let' for variable declarations, 'if' for conditional checks, and 'for' for loops. It also uses 'msg.payload' to access the message data and 'flow.set' to update the flow's state.

```
1  let row = msg.row;
2  if (!row) return null;
3
4  //Packet Type = Link Status (0x08)
5  let packetType = row["Packet Type"];
6  if(!packetType || packetType !== "Link Status (0x08)") return null;
7
8  //ZigBee source address
9  let source = row["Source Address ZigBee"];
10 if (!source) return null;
11
12 //Command String
13 let cmd = row["Command String"];
14 if (!cmd) return null;
15
16 let costs = flow.get("costs") || {};
17 if(!costs[source]) costs[source]={};
18
19 //extract Links section from Command String - 'Links': [{},{},...]
20 let linksMatch = cmd.match(/'Links': \[([^\s]*)\]/);
21 if (!linksMatch) return null;
22
23 //extract all links - [{},{},...]
24 let linksText = linksMatch[1];
25
26 //extract every link {...}
27 let linkItems = linksText.match(/\{[^\}]+\}/g);
28 if (!linkItems) return null;
29
30 //process every link
31 for(let i = 0; i < linkItems.length; i++){
32     let link = linkItems[i];
33     //extract Address
34     let addressMatch = link.match(/'Address': '([^\']+)'/);
35     //extract Outgoing Cost
36     let costMatch = link.match(/'Outgoing Cost': '([^\']+)'/);
37     if (!addressMatch || !costMatch) continue;
38     let destination = addressMatch[1];
39     let cost = Number(costMatch[1]);
40     costs[source][destination] = cost;
41 }
42
43 flow.set("costs", costs);
44
45 msg.payload = {
46     source: source,
47     links:costs[source]
48 };
49
50 return msg;
```

The source address, destination addresses and corresponding outgoing cost values are extracted from ZigBee Link Status packets.

The extracted information is stored inside the flow context for later CSV generation and ThingSpeak processing.

### ➤ generate costs

```
1 let costs = flow.get("costs") || {};  
2 let lines = [];  
3  
4 //outgoing_cost.csv header  
5 lines.push("No.,Source,Destination,Cost");  
6  
7 let no = flow.get("outgoing_no") || 1;  
8  
9 //go through every source and destination to save cost data into CSV lines  
10 for (let source in costs) {  
11     for (let destination in costs[source]) {  
12         let cost = costs[source][destination];  
13         lines.push(`${no},${source},${destination},${cost}`);  
14         no++;  
15         flow.set("outgoing_no", no);  
16     }  
17 }  
18  
19 let csvMsg = {  
20     payload: lines.join("\n")  
21 };  
22  
23 let httpMsg = {  
24     payload: "send_to_thingSpeak"  
25 };  
26  
27 //output 1 -> send generated csv data for outgoing_cost.csv creation  
28 //output 2 -> trigger http/ThingSpeak upload process  
29 return [csvMsg, httpMsg];
```

The generated output is formatted as CSV content using the structure:

No.,Source,Destination,Cost

The formatted data is written into *outgoing\_cost.csv* and simultaneously forwarded to the next process stage.

## ➤ prepare ThingSpeak messages

Name

prepare ThingSpeak messages

Setup

On Start

On Message

On Stop

```
1 let costs = flow.get("costs") || {};  
2  
3 let sources = Object.keys(costs);  
4  
5 if (sources.length === 0) return null;  
6  
7 //sort sources from the smallest from the largest  
8 sources.sort((a, b) => parseInt(a, 16) - parseInt(b, 16));  
9 let smallestSource = sources[0];  
10  
11 //get destinations from the smallest source  
12 let dests = Object.keys(costs[smallestSource]);  
13  
14 //sort destinations from the smallest from the largest  
15 dests.sort((a, b) => parseInt(a, 16) - parseInt(b, 16));  
16  
17 let messages = [];  
18  
19 //prepare ThingSpeak messages with cost data  
20 for (let i = 0; i < dests.length; i++) {  
21     let dest = dests[i];  
22     let cost = costs[smallestSource][dest];  
23  
24     messages.push({  
25         url: "https://api.thingSpeak.com/update?api_key=05ZRLSY3RMPJ6RGA&field1=" + cost,  
26         payload: {  
27             source: smallestSource,  
28             destination: dest,  
29             cost: cost  
30         }  
31     });  
32 }  
33  
34 return [messages];
```

The generated messages contain the smallest hexadecimal source address together with sorted destination addresses and their corresponding outgoing cost values.

Each message is formatted as an HTTP payload for ThingSpeak transmission.

## ➤ limit 1 msg/ 20 sec

Action

Rate Limit

All messages

Rate

1

msg(s) per

20

Seconds

☐ allow msg.rate (in ms) to override rate

Queue intermediate messages

Outgoing ThingSpeak messages are buffered and transmitted sequentially at a fixed rate of one message every 20 seconds.

### ➤ send to ThingSpeak

|        |          |
|--------|----------|
| Method | GET      |
| URL    | https:// |

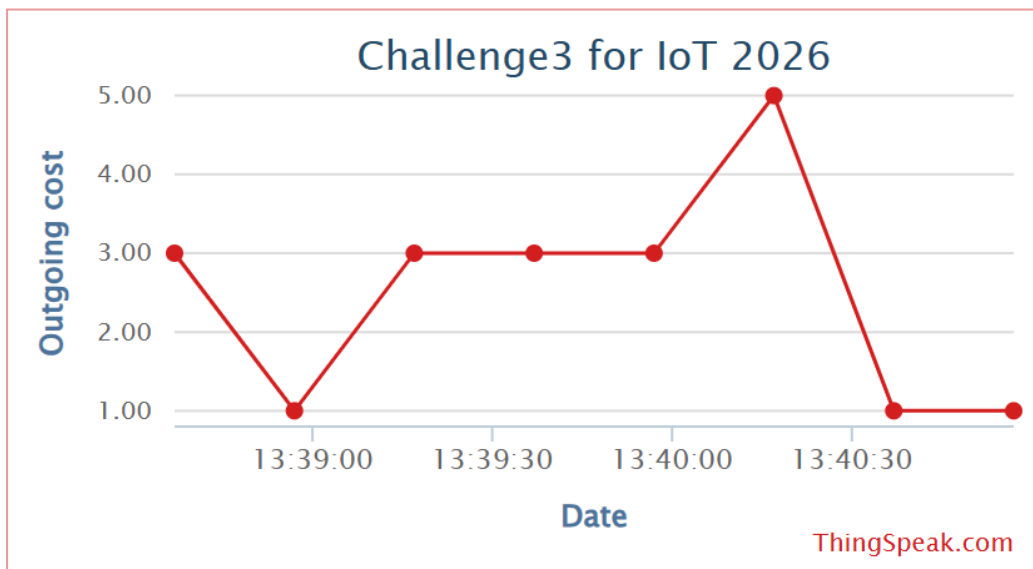
The complete ThingSpeak request URL is dynamically generated in the previous Function node using *msg.url*.

## 5.2 ThingSpeak Output Visualization

The figure shows the sorted destination addresses and their corresponding outgoing cost values for the smallest source address (0x0000).

|    |        |        |   |
|----|--------|--------|---|
| 33 | 0x0000 | 0x0112 | 3 |
| 34 | 0x0000 | 0x1cd8 | 1 |
| 35 | 0x0000 | 0x1e15 | 3 |
| 36 | 0x0000 | 0x3d95 | 3 |
| 37 | 0x0000 | 0x4e11 | 3 |
| 38 | 0x0000 | 0xa706 | 5 |
| 39 | 0x0000 | 0xe1a6 | 1 |
| 40 | 0x0000 | 0xec7f | 1 |

The figure shows the outgoing cost values transmitted to the ThigSpeak channel.



It is possible to view it through the following link: [Challenge3 for IoT 2026 - ThingSpeak IoT](#).